

Programming for Artificial Intelligence

Individual Assignment

Takuya Otsuki

Task 2

1. Justification of Design

1.1 Data structure selection

The project employs an undirected weighted graph as the main data structure which is represented by using an adjacency list with dictionaries. Each node represents a product, and edges represent co-purchase relationships. Weights represent frequency of co-purchase. This design was selected as it is efficient when data is sparse. Furthermore, the graph structure can express products co-purchased with multiple other products.

The space complexity of the adjacency list is $O(V+E)$, where V represents the number of unique items and E is the number of co-purchase relationships. In this dataset, since $V = 167$ and $E = 6260$, an adjacency list requiring $O(V+E)$ is more efficient than an adjacency matrix requiring $O(V^2)$. Dictionaries provide $O(1)$ for edge weights and neighbor retrieval.

1.2 Algorithm implementation and complexity analysis

Four algorithms were built:

Graph construction Algorithm: Time complexity is $O(n * m^2)$, where n represents the number of transactions (14963) and m represents the average number of purchased items per transaction (2.59). For each transaction,

Co-purchase query algorithm: This algorithm retrieves neighbors for a specific item and its complexity is $O(k \log k)$, where k represents the number of the neighbors. The process is retrieving neighbor using dictionaries with $O(k)$, filtering with $O(k)$, and sorting with $O(k \log k)$.

Top bundles Algorithm: This algorithm extracts the most frequent pair by iterating all the edges with $O(E)$ and sorting them with $O(E \log E)$.

BFS for related items search: it uses BFS and time complexity is $O(V+E)$ and space complexity is $O(V)$. `max_depth` defines the exploration depth.

1.3 Alternative Approaches

As an alternative way to extract the frequent items, association rule mining algorithms like Apriori (Noble, nd) can be considered. But it will explore every possible item combination with complexity $O(2^m)$, resulting in enormous amount of calculation. On the other hand, the graph approach applied to this project conducts pair processes with complexity $O(m^2)$ per transaction. So, the graph one is more realistic. Moreover, it enables using graph algorithms like BFS, making it possible to explore relationships between items beyond co-occurrence.

2. Evaluation and Scalability

2.1 Performance with larger datasets

This implementation can efficiently process the dataset with 14963 transactions and 167 unique items. The complexity of the graph approach is linear against the number of transactions, so that is $O(n)$, while it comes to basket size, it depends on complexity $O(m^2)$. Currently, since the average basket size is just 2.9, the effect is not so considerable in the current dataset.

If it is assumed that transaction grows to ten times with a similar average basket size, construction time will proportionally increase. However, if basket size were considerably bigger, the construction time would become worse quadratically. In this case, the construction time would 100 times longer.

Memory usage increase by following $O(V+E)$. The number of edges does not necessarily increase in linear with transactions because the combinations of popular items like milk and bread are repeatedly observed and it will increase the weights on the existing edges.

2.1 Dynamic Dataset Challenges

The implementation is designed to load all the data when it is initialised, which is suitable for batch analysis but not for real-time updates. Adding new transactions can be processed at $O(m^2)$ by using `add_transaction`, but the function to eliminate transactions is not implemented. To implement the delete function, tracking which transaction contributes to each edge weight is needed and it causes extra memory overhead.

For the circumstances where transactions are generated continuously like actual supermarket, it is not enough to reconstruct the whole graph and it is needed to update the graph incrementally. The current structure can handle additions, but not support deletion of item or transaction cancellations. So, it would need to be modified in that case.

2.3 Real-world applicability

The system is useful for use cases of practical retail analysis. The co-purchase query can optimise the item layout in stores based on the frequent co-purchase pair. The top bundles algorithm can make better promotional strategies by figuring out common purchasing patterns. And the BFS algorithm can identify indirect item relationships.

On the other hand, there are some constraints when it comes to large scale deployment. Since this system stores all the data in memory, it is not suitable for huge retailers with a number of purchases occurring across multiple places. Additionally, this does not cover the seasonal trends or chronological trends. To implement this functionality, multiple graphs would be needed.

This system is focusing on precision and readability over optimisation, so suitable enough for analysis purposes or decision-making.

65 tests were passed on TDD and the design is well-prepared for future extensions.

Reference list

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. Available at:

https://books.google.co.uk/books?hl=en&lr=&id=vqTfNFDzzdIC&oi=fnd&pg=PR7&dq=Patterns+of+Enterprise+Application+Architecture&ots=oVF1GwOSrw&sig=ad8pBYQpn3CYqQPWE6b139ls0Gg&redir_esc=y#v=onepage&q=Patterns%20of%20Enterprise%20Application%20Architecture&f=false

GeeksforGeeks (2025a) *DRY Principle in Software Development*. Available at:

<https://www.geeksforgeeks.org/software-engineering/dont-repeat-yourselfdry-in-software-development/> (Accessed: 15 December 2025).

GeeksforGeeks (2025b) *MVC Design Pattern*. Available at:

<https://www.citethemrightonline.com/sourcetype?docid=b-9781350927964&tocid=b-9781350927964-34> (Accessed: 15 December 2025).

GeeksforGeeks (2025c) *Requirements Traceability Matrix – RTM*. Available at:

<https://www.geeksforgeeks.org/software-testing/requirement-traceability-matrix/> (Accessed: 15 December 2025).

Noble, J (no date) *What is the Apriori algorithm?*. Available at:

<https://www.ibm.com/think/topics/apriori-algorithm> (Accessed: 15 December 2025).

van Rossum, G., Warsaw, B. and Coghlan, A. (2001) *PEP 8 – Style Guide for Python Code*.

Available at: <https://peps.python.org/pep-0008/#introduction> (Accessed: 15 December 2025).